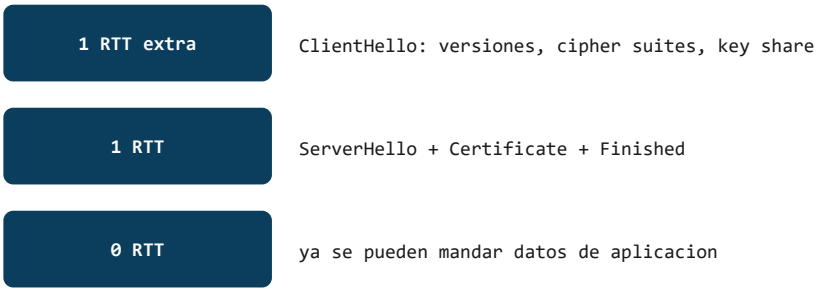
Clase 47 — Seguridad: servicios y criptografía

Bloque: F (Servicios y gestión) · **Semana:** 16 · **Duración estimada:** 130 min **Correlativa:** Clase 46 (gestión de redes) · **Sigue con:** Clase 48 (seguridad aplicada) **Material de referencia:** Tanenbaum cap. 8 · `ACTUALIZACIONES.md` §13

🎯 Al terminar esta clase vas a poder

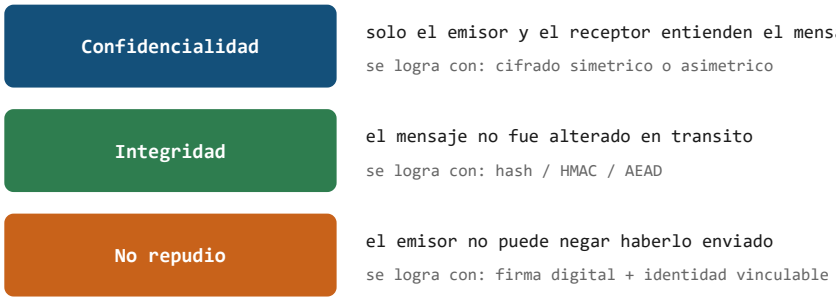
1. Distinguir **confidencialidad**, **integridad**, **autenticidad** y **no repudio**, y decir **qué técnica** consigue cada una y cuál **no**.
2. Explicar qué es una **función de hash**, enumerar sus **cuatro propiedades**, y decir por qué **el hash da integridad pero NO confidencialidad**.
3. Explicar por qué **MAC no sirve para firmar** y por qué **el hash solo tampoco**.
4. Describir **RSA** y la razón matemática que lo sostiene, y sus **tres limitaciones** (lento, tamaño, y no firma directamente).
5. Describir la **firma digital** paso a paso y explicar **por qué se firma el hash y no el mensaje**.
6. Describir una **PKI**: qué es un certificado, qué es una cadena de confianza, qué es el **ancla**, y **por qué todo el modelo descansa en un supuesto**.
7. Escribir el esquema **híbrido** de un canal seguro real (lo que hace TLS).

Handshake de TLS 1.3 en un RTT



TLS 1.3 cambio: solo AEAD, sin compresion, sin renegociacion, y forward secrecy obligatorio.
El certificado sigue siendo X.509; lo que cambio fue el intercambio de claves.

Los tres servicios de seguridad



> **La diferencia fina entre integridad y no repudio NO es el algoritmo.**
> Es QUIEN controla la clave privada: si la tiene solo Alicia, Alicia no puede negar.

Y los tres se resuelven juntos en TLS, no por separado.

Sumalos: autenticacion (soy quien digo ser) y control de acceso.

> **Prerequisito del examen:** `2010-02 P4` (8 pts — firma digital y por qué MAC no puede firmar), `2009-12 P4` (6 pts — funciones de hash y propiedades de seguridad), `2009-02 P1` (9 pts — TLS: qué clave va en el certificado, y cuál protege los datos), `2008-12 P2` (seguridad en switch vs. hub).

PARTE 1 — TEORÍA

1.1 Los cuatro servicios (y el confundón de siempre)

Antes de cualquier algoritmo hay que tener esta tabla **clara**, porque el 80 % de los errores en un examen salen de confundir los servicios.

Servicio	Pregunta que responde	Ejemplo
Confidencialidad	¿puede alguien **leerlo**?	cifrar una carta
Integridad	¿puede alguien **cambiarlo**?	el sello de lacre
Autenticidad	¿es **realmente de vos**?	tu firma manuscrita
No repudio	¿podés **probar** que fuiste vos?	firma notarial

Y las dos relaciones que hay que tener presentes porque son las que se preguntan:

```
"Si NO tengo confidencialidad, ¿puedo tener integridad?"

     SÍ. Ejemplo: un disco cifrado donde te avisan SÓLO si un archivo cambió
          (hash cifrado con HMAC). Vos no podés leer el contenido, pero
          sabés si alguien lo toco.
     ★ Por eso "confidencialidad" NO implica "integridad", ni al revés.
       Son servicios INDEPENDIENTES.

"Si tengo confidencialidad por cifrado SIMÉTRICO, ¿tengo autenticidad?"

     Depende. El cifrado simétrico por sí solo NO autentica al emisor:
     cualquiera que conozca la clave puede cifrar algo haciéndose pasar
     por vos. Para autenticar hace falta un MAC o un modo AEAD.
```

La taxonomía de las técnicas

```
┌──────────────────────────────────────────────────────────────┐
│  CIFRADO (en claro, con clave)                               │
│    simétrico   → misma clave para cifrar y descifrar         │
│                  (AES, ChaCha20)                             │
│    asimétrico  → par de claves: una pública, una privada     │
│                  (RSA, ECC / Ed25519)                        │
│                                                              │
│  HASH (sin clave, una sola vía)                              │
│    → NO cifra, NO tiene clave                                │
│    → detecta cambios (integridad), no oculta nada            │
│    → SHA-256                                                 │
│                                                              │
│  MAC (con clave simétrica, integridad + autenticación)       │
│    → HMAC-SHA-256                                            │
│    → detecta cambios Y sabe quién lo hizo, pero NO cifra     │
│                                                              │
│  FIRMA (asimétrica, integridad + autenticidad + no repudio)  │
│    → firma sobre el hash, con clave privada                  │
│    → RSA-PSS, ECDSA, Ed25519                                 │
│                                                              │
│  CERTIFICADOS / PKI                                          │
│    → vinculan una clave pública con una identidad            │
│    → X.509                                                   │
└──────────────────────────────────────────────────────────────┘
```

1.2 La función de hash

Función de hash = una función que toma cualquier mensaje (de cualquier tamaño) y devuelve una cadena **de tamaño fijo**.

```
m  ──────────►  H(m)  ──────────►  h
(cualquier          (la función)     (huella: siempre
 tamaño)                              32 bytes con SHA-256)

m = "hola"          →  h = 2cf24dba...
m = "hola "  (con   →  h = 5891b5b...   ← ¡distinto! Un bit cambia TODO
un espacio)
m = "holb"          →  h = aa4b0c95...      el hash.
```

Propiedades obligatorias (si una falla, la función no sirve para seguridad):

#	Propiedad	Qué dice	Por qué importa
1	**Determinista**	la misma entrada da **siempre** la misma salida	si no, no podrías verificar
2	**Unidireccional** *(preimage resistance)*	dado `h`, es **computacionalmente inviable** encontrar `m` tal que `H(m) = h`	sin esto, cualquiera que vea la huella puede **reconstruir el mensaje**
3	**Sin colisiones** *(second preimage)*	dado `m`, es inviable encontrar **otro** `m'` con `H(m') = H(m)`	sin esto, podés **reemplazar** el mensaje por otro con la misma huella
4	**Avalancha**	cambiar **1 bit** de la entrada cambia **~50 %** de los bits de la salida	hace imposible el ataque de **cota incremental**

Y dos propiedades "de facto" que se piden:

- **Longitud de salida fija y suficiente**: 256 bits con SHA-256. Con 128 bits (MD5, SHA-1) la probabilidad de **colisión** por fuerza bruta es $2^{-64} \approx 5\times10^{-20}$; con 256 bits es $2^{-128} \approx 3\times10^{-39}$.
- **Resistencia a preimagen de segundo orden**: no es lo mismo encontrar **una** colisión (cualquier par `m, m'`) que encontrar una colisión **para un** `m` **dado** (second-preimage). La segunda es más difícil, y SHA-256 resiste las dos.

El hash NO es cifrado. Es unidireccional: no hay "descifrar" un hash. Y **no lleva clave**: la misma función, para todos, siempre. Es por eso que un hash "público" sirve para cualquier cosa: no se puede usar para ocultar, solo para detectar.

Y ojo con el orden de la frase en un examen: "*un hash provee integridad pero no confidencialidad*". Si escribís al revés, es un error conceptual, no de redacción.

⚠️ MD5 y SHA-1: la historia que hay que saber

```
MD5      128 bits. ROTO desde 2004 (ataques de colisión hallados).
         En 2008, el "rogue CA": una CA legitimamente firmada un
         certificado con la MISMA huella que otro, porque MD5 ya era
         vulnerable. El sistema de confianza entero colapsó.
SHA-1    160 bits. ROTO en 2017 con SHAttered (2<sup>63</sup> intentos, 6500
         años de un core → 110 años con 6500 GPUs y 110 $/GPU-hora).
SHA-256  256 bits. SEGURO.
SHA-3    256/512. SEGURO, y con diseño diferente a SHA-2.

RFC 9155 (2021) → depreca explícitamente MD5 y SHA-1 en TLS 1.2
```

1.3 Cifrado simétrico

```
MENSAJE ──[ CIFRAR con k ──► CIFRADO ──[ DESCIFRAR con k ──► MENSAJE
                (la MISMA clave k)
```

Característica esencial: es rápido y hay que compartir la clave. Y compartir la clave es el problema: en un canal de N personas, hacen falta **N−1 claves**.

El problema de la distribución de claves es tan grave que la criptografía simétrica sola **no resuelve** nada en un canal abierto: si vos y yo queremos hablar y hay un tercero escuchando, tenemos que ponernos de

acuerdo en la clave **sin que el tercero se entere**. Y ese es exactamente el problema que resuelve la criptografía asimétrica.

Los modos de operación: por qué importa

Cifrar "cada bloque por separado" con ECB es **inseguro**, y el motivo es importante para entender por qué:

```
ECB  (Electronic Codebook)
──────────────────────────

Bloque1 → C1
Bloque2 → C2          Los bloques IGUALES producen el MISMO
Bloque3 → C1          cifrado. Consecuencia: se ve el PATRÓN
Bloque4 → C2          del texto original a través del cifrado.
                     ★ Famous: la imagen del "dinosaurio".
```

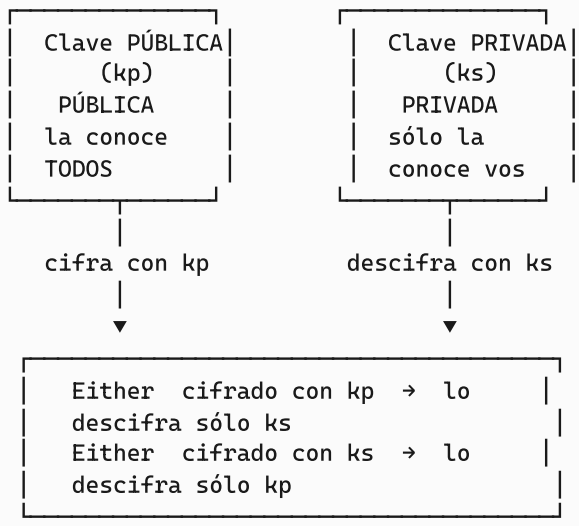
Las soluciones: **CBC** (cifrado en cadena), **CTR** (contador, como un cifrado de flujo), y el estándar moderno: **AEAD** (*Authenticated Encryption with Associated Data*), que **junta cifrado e integridad en una sola operación**:

```
AES-GCM          →  cifrado + integridad (y el tag de autenticación)
ChaCha20-Poly1305 →  lo mismo, más rápido sin acelerador de hardware
```

> **El AEAD es hoy el default** y la razón tiene nombre: **evita los ataques de manipulación**. Con "cifrar y después hashear por separado" (o peor, hashear por separado *antes* de cifrar) se abre la puerta a ataques de **cambio de bits** y a **padding oracle** (el famoso ataque de Vaudenay contra TLS). El AEAD calcula la autenticación **dentro** del cifrado, así que no hay forma de tocar el texto sin romper el tag.

1.4 Cifrado asimétrico: RSA

El cambio de paradigma: en vez de una clave, un **par**.



Lo que faltaba: la clave privada **nunca viaja**. Aunque alguien intercepte mil millones de mensajes, no puede sacar la clave privada **de los mensajes**, porque es un problema **matemático**, no de ingeniería.

La base matemática (lo mínimo que hay que saber)

RSA usa el problema de **factorización de enteros grandes**:

```
1. Se eligen DOS primos GRANDES:  p  y  q
2. n = p · q                                    ← el MÓDULO (es público)
3. ϕ(n) = (p−1)(q−1)                            ← función Euler
4. se elige e  tal que   gcd(e, ϕ(n)) = 1       ← EXPONENTE PÚBLICO
5. d  = e⁻¹ mod ϕ(n)                            ← EXPONENTE PRIVADO
6. El secreto es (p, q); el resto es público: (n, e)
```

Y funciona por el **teorema de Euler**:

```
Para cualquier m:      m^(e·d) ≡ m  (mod n)

O sea:  cifro con e, descifro con d → obtengo el mensaje original.
Y para romperlo tendría que factorizar n (o resolver el problema de
RSA directamente, que es equivalente a factorizar).
```

El problema de factorización es believed hard: hoy se han roto claves de **829 bits**, y las factorizaciones de RSA-2048 (que es lo que se usa) están fuera de alcance con la computación actual. **Pero — y esto es lo importante — no hay prueba de que sea hard**: nadie ha demostrado que factorizar sea difícil, solo que **parece**.

Y las tres limitaciones de RSA, que se preguntan siempre:

Limitación	Detalle
Es lento	Modular exponenciación: la operación cuesta **$O(k^3)$** en el tamaño de la clave. **AES va 100 a 1000 veces más rápido.**
Los datos son gigantes	Con claves de 2048 bits solo podés cifrar **245 bytes**. Para cifrar un archivo de 1 MB, lo dividís, cifrás cada parte y las concatenás (el esquema **híbrido**).
No cifra en bloque	RSA solo cifra bloques del tamaño del módulo. No es un cifrado de flujo ni de bloque utilizable para datos largos.

> **Y un cuarto problema, más moderno:** la firma directa con RSA (`PKCS#1 v1.5`) es **frágil** a ataques de Bleichenbacher. En TLS 1.3, el **transporte de clave con RSA fue directamente eliminado**: hoy solo queda **RSA-PSS**, que es el modo de firma moderno.

ECC y Ed25519

RSA **no es lo más seguro por bit**. Con la seguridad de 128 bits (la de AES-128):

```
RSA       necesita  ~3072 bits
ECC       necesita  ~256  bits       ← misma seguridad, clave 12× más corta
```

Ventajas de ECC en la práctica: claves más cortas = ** menos cómputo, menos bandwidth en el handshake, menos batería **(crítico en móviles y en IoT).** Ed25519 **es la curva más usada hoy (es el default de** SSH**, `RFC 8709`).

La tabla de los tres servicios que hay que saber:

Algoritmo	Para qué	Clave pública	Clave privada
RSA	cifrar, firmar	`(n, e)`	`d`
ECDSA / Ed25519	**firmar** (no cifra bien)	punto en la curva	escalar
ECDH / X25519	**acuerdo de clave**	punto en la curva	escalar

> **El matiz de examen:** ECDSA **firma**, no cifra; Diffie-Hellman **acuerda**, no cifra ni firma. Confundirlos es el error clásico.

1.5 El hash no firma solo: por qué

Este es **el corazón conceptual** de la clase, y da exactamente el examen `2010-02 P4`.

```
ARGUMENTO (imposible usar hash solo para firma):

1. El hash NO lleva clave. Cualquiera puede calcular H(m).
2. Si "la firma fuera H(m)", el atacante también puede calcular
   H(m) y ponerlo al lado del mensaje.
3. Entonces CUALQUIERA podría "firmar" cualquier mensaje.
   ★ No hayNpseudoLfirma.

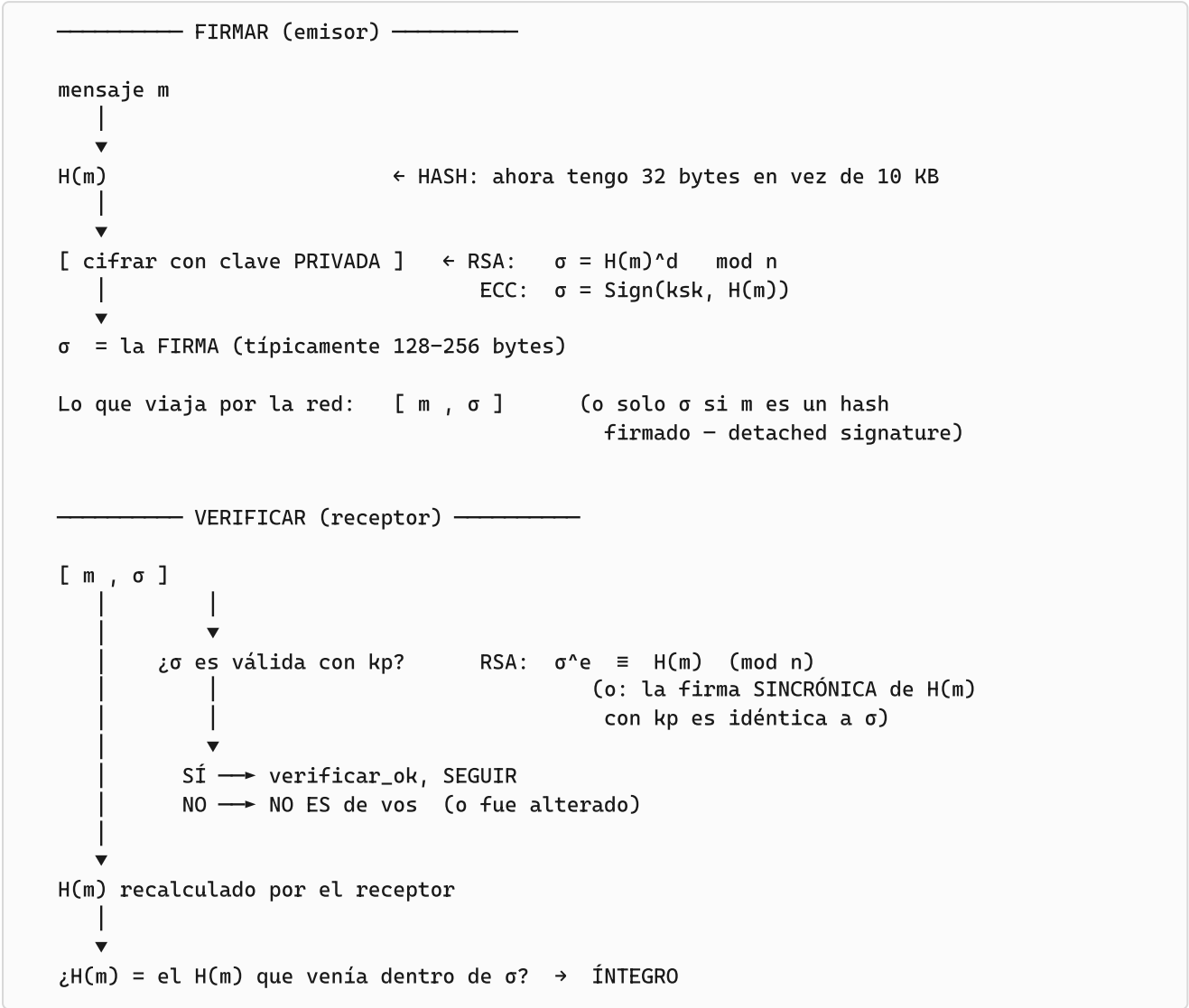
ARGUMENTO (imposible usar MAC para firma):

1. El MAC usa clave SIMÉTRICA (HMAC).
2. Para verificar, el receptor necesita la MISMA clave.
3. Para poder verificar, vos tenés que darle tu clave a todos.
4. Si todos tienen la clave, TODOS pueden firmar como vos.
5. ★ No hay "solo vos" podés firmar: no hay no-repudio.

SOLUCIÓN: combines asimetría + hash.
──────────────────────────────────────
1. El hash sigue dando INTEGRIDAD (detecta cualquier cambio).
2. La clave PRIVADA da AUTENTICIDAD (solo vos podés firmar).
3. La clave PÚBLICA da VERIFICABILIDAD sin secreto.
4. ★ La combinación da los 4 servicios a la vez.
```

> **La frase que hay que saber de memoria:** *"el hash detecta el cambio pero no dice quién lo hizo; el MAC detecta el cambio y sabe quién lo hizo, pero todos pueden hacerlo; la firma detecta el cambio, sabe quién lo hizo, y es la única forma de probar que fue esa persona y no otro porque solo esa persona tiene la clave privada que no se puede compartir."*

1.6 La firma digital, paso a paso



Por qué se firma el hash y no el mensaje — y esta es la respuesta que más puntúa:

1. **Eficiencia.** RSA tarda milisegundos. Hashear un archivo de 1 GB tarda microsegundos. Si tuvieras que **firmar el mensaje entero**, la operación criptográfica costaría **lo mismo** que hashear + firmar. Con el hash, firmás 32 bytes.
2. **La firma queda en tamaño fijo**, sea cual sea el mensaje (32 bytes de hash → firma de 256 bytes). Un certificado X.509 firma una lista de SAN: si el mensaje fuese enorme, la firma sería gigante.
3. **La longitud no filtra el secreto.** Con el **padding** determinista, si cifrás el mensaje directamente, el tamaño del cifrado **revela el tamaño del mensaje**. Con un hash de longitud fija, no.

⚠️ **Un matiz que hay que tener claro:** en la práctica, la firma digital **no cifra** el mensaje. El mensaje viaja **en claro** junto a la firma. Solo se **cifra** si querés confidencialidad, y para eso se usa el **esquema híbrido**.

El esquema híbrido: cómo se hace en la realidad

Ninguno de los dos algoritmos sirve solo. Entonces se combinan:

```
── Cifrar un archivo de 1 MB ──

1. Generar una clave de sesión ALEATORIA K          ← 256 bits, efímera
2. Cifrar el archivo con K          (rápido, simétrico, AES-GCM)
3. K se envía... ¿cifrada con RSA? NO. Con el ACUERDO de clave (KEM):
    a. El receptor publica kp.
    b. Vos hacés KEM(kp) = tu secreto + su secreto → K_secreto compartido
    c. Derivás la clave de sesión con un KDF (HKDF)
4. Se transmite:  [archivo cifrado] + [tu secreto encapsulado]
5. El receptor descifra, aplica el KDF, obtiene K, descifra el archivo.

★ Confidencialidad: SIMÉTRICA (rápida) sobre datos largos
★ Autenticación: ASIMÉTRICA sobre la clave corta
★ El archivo nunca sale de claro en el cable ✓
```

Ése es el diseño de TLS, de PGP, de IPsec, de SSH, de todo. Y es la razón por la que **RSA no se usa para cifrar datos**: el costo lo pone la clave, que es corta.

1.7 PKI: el problema de la confianza

Hash, cifrado y firma **ya funcionan**. El problema que queda:

```
Beto le manda a Ana un mensaje firmado con ksb.
Ana tiene kpb "de Beto"... ¿de dónde la sacó?

Si alguien le pasa una clave PÚBLICA por Internet y dice "esta es de Beto",
Ana no tiene forma de saberlo. El atacante le pasa kpa y dice "esta es
de Beto", intercepta y firma como Beto.
★ Mismo problema: falta una raíz de confianza.
```

La solución: el certificado y la cadena de confianza.

```
                RAÍZ  (Root CA)
            auto-firmada, se confía en ella "de fábrica"
             │         │         │
             │         │         └──────── R3 (autoridad de nivel 3)
             │         └──────────────────── R2 (nivel 2)
             └──────────────────────────────── R1 (intermedia)

                │ firma a
                ▼
            CA INTERMEDIA
                │ firma a
                ▼
        CERTIFICADO DE HOJA  ("el certificado del servidor")
        ──────────────────────────────────────
        · sujeto:  CN = www.miuy.edu.uy
        · emisor:   la intermedia
        · kp:       la clave pública de www.miuy.edu.uy
        · validez:  desde / hasta
        · Serial,    firma de la intermedia
          algoritmo, huella,   ← el hash del certificado (firmado)
          SAN
        ──────────────────────────────────────
```

La cadena de confianza: para validar el certificado de `www.miuy.edu.uy`, el cliente sube: hoja → intermedia → raíz. Y **cada eslabón firma al siguiente**. Si llega a una raíz que el cliente tiene en su **almacén de confianza** (*trust store*), la cadena es válida.

> **Y acá está el punto conceptual que hay que decir en el examen:** *toda la seguridad de este modelo descansa en un único supuesto: que las raíces del almacén de confianza son legítimas. Si meten una raíz falsa en tu sistema, todo lo demás se deriva de ella y nada te protege. Ese supuesto es el punto más débil de la PKI, y es la razón por la que existe DNSSEC (ancla en la raíz DNS) y RPKI (ancla en la jerarquía de prefijos).*

Y `SAN` (*Subject Alternative Name*) es lo que hay que saber hoy: el **CN** (*Common Name*) —que era "el nombre del sitio"— **ya no se usa** para validar identidad (Chrome y Firefox lo rechazan). Lo que se valida es el campo **SAN**, que puede contener **DNS names** e **direcciones IP**.

1.8 ⚠️ Actualización a 2026

Sigue igual — de memoria: - **Hash**: SHA-256. Nunca MD5 ni SHA-1. - **Simétrico**: AES-GCM / ChaCha20-Poly1305. AEAD es el default. - **Firma**: RSA-PSS, ECDSA, Ed25519. - **Key agreement**: (EC)DHE con X25519, o híbrido ML-KEM. - **PKI X.509**: cadena AC → intermedia → hoja, validación, `SAN` (no CN). - **Forward secrecy** con (EC)DHE. **HKDF** para derivar claves. - **DH estático NO da forward secrecy** (Logjam) — hay que usar ephemera.

Cambió:

1. **RSA key transport eliminado en TLS 1.3** (`RFC 8446`). Solo queda firma con **RSA-PSS**; `PKCS#1 v1.5` es legacy.
2. **ML-KEM híbrido** (`RFC 10024` , ago-2026) es el nuevo key agreement de facto para resistencia post-cuántica.
3. **SHA-1 formalmente retracted.** NIST lo retiró para firmas en 2011; SHAttered (2017) halló colisiones prácticas; `RFC 9155` (2021) depreca MD5 y SHA-1 en TLS 1.2.
4. **Ed25519 es el default en SSH** (`RFC 8709`) y creciente en X.509.
5. **HTTP Message Signatures** (`RFC 9421` , 2024): firma de *componentes* de un mensaje HTTP, resistente a transformaciones de intermediarios.
6. **OpenPGP:** `RFC 9580` (2024) reemplaza a RFC 4880.

Y la lección conceptual de 2026, que es lo que hay que saber decir:

> **La integridad y la autenticidad son *relativas a un ancla de confianza*, y hay un ancla distinta por ámbito.** Un curso que solo enseña "integridad = HTTPS" se queda cortísimo:

Ámbito	¿Qué es el ancla?	RFC
Transporte web	**PKI / CA** (X.509)	`RFC 8446` → `9846`
DNS	**DNSSEC** + Root KSK	`RFC 4033–4035`, `5155`
Enrutamiento	**RPKI** / ROA	`RFC 6811`
Capa 2	**802.1X** / MAB, Port Security	—
Email	S/MIME u **OpenPGP**	`RFC 9580`
Código / binarios	**firma de commits** (GPG/Sigstore)	—

El no repudio real es raro y costoso: requiere clave privada controlada por el firmante **más** identidad vinculable. **DNSSEC y RPKI son no-repudio de *infraestructura*** — dicen quién originó un nombre o un prefijo, **no quién mandó un correo**.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — Qué servicio consigue cada técnica

Enunciado. Completar la matriz con ✓ / ✗.

Resolución:

	Confidencialidad	Integridad	Autenticidad
Hash (SHA-256)	✗ NO oculta nada	✓ detecta cambio	✗ cualquiera puede hashear
MAC (HMAC)	✗	✓	✓ con clave
Cifrado sim. (AES-ECB)	✓ (y nada más)	✗	✗ cualquiera con la clave cifra
Cifrado sim. en modo AEAD (AES-GCM)	✓	✓	✓ (con clave)
Cifrado asim. (RSA)	✓ (por partes)	✗	✓ solo el dueño de la priv
Firma digital (RSA-PSS/Ed)	✗ NO cifra el mensaje	✓	✓ con clave priv
+ No repudio (firma + identidad)	✗	✓	✓ Y se puede PROBAR
TLS 1.3 (esquema híbrido)	✓	✓	✓ + FS

Las tres filas que se explican siempre:

- **Hash**: detecta **que cambió**, no **quién** cambió. Y no oculta nada.
- **Cifrado simétrico puro (ECB)**: esconde el contenido, pero cualquiera con la clave puede cifrar cualquier cosa y **no se detecta la manipulación**.
- **Firma digital**: **no cifra**. El mensaje va en claro; lo que va es la firma. Confundir esto es el error más común.

2.2 Ejercicio 2 — La cadena completa de verificación de un certificado

Datos. El navegador abre `https://www.miuy.edu.uy`.

Resolución paso a paso:

```
1. Obtener el certificado
   El navegador hace el TLS handshake; el servidor le manda su
   certificado de hoja (ya en claro, no cifrado).

2. Extraer la clave pública
   Del certificado sale kp_srv = (n, e).   ← ESTE es el dato útil.

3. Validar la FIRMA del certificado
   El certificado viene firmado por la intermedia.
   Se verifica σ_int con kp_int:
      σ_int^e_int  mod n_int   debe dar   H(certificado)
   Si no coincide → el certificado fue alterado. ✗

4. Validar la CADENA
   Extraer kp_int del certificado de la intermedia.
   Ese certificado está firmado por la raíz.
   Se verifica, y así sucesivamente hasta llegar a una raíz.

5. Validar el ANCLA
   ¿La raíz está en el TRUST STORE del sistema?
      (Windows: almacén de certificados. Mac: login keychain.
       Firefox/Chrome: los suyos + los del SO.)
   SÍ → la cadena es de fiar.   NO → ✗ no confío.

6. Validar la IDENTIDAD
   Extraer el SAN del certificado de hoja.
   ¿"www.miuy.edu.uy" está en el SAN?
      SÍ → ✓.   NO → ✗ (aunque la cadena sea válida:
                    el certificado es de otro sitio!)

7. Validar la VIGENCIA
   ¿Estamos dentro del período de validez?
   ¿La revocación no está activa?  (CRL u OCSP)

8. Solo ahora: usar la clave
   Se cifra la clave de sesión con kp_srv  → se abre el canal.
```

> **El paso 6 es el que más se olvida y el más importante conceptualmente:** una cadena válida **no garantiza identidad**. Lo que garantiza es que **alguien firmó** ese certificado. Hay que comprobar **que firmó del nombre correcto**. Por eso un certificado de `evil.com` reemitido por una CA real es **cadena válida con identidad incorrecta**, y es lo que valida el SAN.
>
> Y el paso 8 es donde aparece la **moderna**: en TLS 1.3 **no se cifra nada con RSA**. Se usa la clave del certificado para **firmar** el *handshake* (autenticar al servidor), y la clave de sesión se acuerda con (EC)DHE.

2.3 Ejercicio 3 — Cifrado híbrido, números a la vista

Datos. Un archivo de 3 MB. AES-256 a 2 GB/s. RSA-2048 cifra 245 bytes por operación, a **300 operaciones/s**.

Resolución — las dos cuentas:

```
── Con RSA solo ──
3 MB = 3 × 1024 × 1024 = 3 145 728 bytes
Bloques de 245 bytes  = 3 145 728 / 245 = 12 839 operaciones
Tiempo = 12 839 ops / 300 ops/s = 42,8 segundos

★ 43 SEGUNDOS para cifrar 3 MB. Inusable.

── Con AES solo ──
3 145 728 bytes / 2 × 10⁹ bytes/s = 0,0016 s = 1,6 milisegundos

★ 26 000 veces más rápido.

── Con el esquema híbrido ──
1. K = 256 bits aleatorios                              →  ~0
2. Cifrar el archivo con AES-GCM usando K  →  1,6 ms
3. KEM(kp_destinatario): encapsular K        →  ~2 ms (una operación RSA)
4. Total ≈ 3,6 ms   ← DOMINADO por la criptografía de clave pública,
                       pero el dato pesado ya está cifrado.
```

La moraleja, que es la que hay que escribir: *la clave pública cifra una sola clave de sesión corta; el simétrico cifra los datos largos. Por eso todo protocolo moderno es híbrido y por eso la clave pública tiene que ser grande.*

2.4 Ejercicio 4 — Módulo RSA y cifrado/descifrado numérico

Datos. `p = 61`, `q = 53`. `e = 17`. Mensaje `m = 65`.

Resolución paso a paso:

```
1. n = p · q = 61 × 53 = 3233
2. ϕ(n) = (p−1)(q−1) = 60 × 52 = 3120
3. e = 17   ( gcd(17, 3120) = 1  ✓ )
4. d = e⁻¹ mod ϕ(n)

   Buscamos d tal que   17 · d ≡ 1  (mod 3120)
   Extendido euclidiano:

      3120 = 183·17 +  9
        17 =   1·9  +  8
         9 =   1·8  +  1
         8 =   8·1  +  0

   De abajo hacia arriba:
         1 = 9 − 8
           = 9 − (17 − 9)
           = 2·9 − 17
           = 2·(3120 − 183·17) − 17
           = 2·3120 − 367·17

   ⇒  17⁻¹ mod 3120 = −367 ≡ 3120 − 367 = 2753
   ⇒  d = 2753

VERIFICACIÓN:  17 × 2753 = 46 801
               46 801 / 3120 = 15,0 exacto   (15 × 3120 = 46 800, +1) ✓

5. Clave PÚBLICA  = (n, e) = (3233, 17)
   Clave PRIVADA = (d)   =  2753
```

Cifrado:

```
c = m^e mod n = 65^17 mod 3233

65² mod 3233 = 4225 mod 3233 = 992
65⁴ mod 3233 = 992² = 984 064 mod 3233

   984 064 / 3233 = 304, resto:
   3233 × 304 = 982 832
   984 064 − 982 832 = 1 232
⇒  65⁴ ≡ 1232 (mod 3233)

65⁸ mod 3233 = 1232² = 1 517 824 mod 3233
   3233 × 469 = 1 516 277
   1 517 824 − 1 516 277 = 1 547
⇒  65⁸ ≡ 1547 (mod 3233)

65¹⁶ mod 3233 = 1547² = 2 393 209 mod 3233
   3233 × 740 = 2 392 420
   2 393 209 − 2 392 420 =  789
⇒  65¹⁶ ≡ 789 (mod 3233)

65¹⁷ = 65¹⁶ × 65 = 789 × 65 = 51 285 mod 3233
   3233 × 15 = 48 495
   51 285 − 48 495 = 2 790

c = 2790
```

Descifrado:

```
m = c^d mod n = 2790^2753 mod 3233

Por el teorema de Euler (e·d = 46 801 = 15·3120 + 1):
     c^(e·d) ≡ c^(1 + 15·ϕ(n))  ≡ c · (c^ϕ(n))^15  ≡ c · 1

Es decir:  2790^46 801 ≡ 2790 (mod 3233)
Y como 46 801 = 2753 × 17 :
     2790^2753 ≡ 65   (mod 3233)   ← ¡es el mensaje original!

m = 65  ✓
```

> ⚠️ **Errores típicos en este ejercicio:** - Confundir `ϕ(n)` con `n`: RSA necesita `ϕ(n)`, no `n`. - Olvidar el **extendido euclidiano** y quedarse sin `d`. Se puede buscar por fuerza bruta `d` desde 1 hasta `ϕ(n)`, pero eso es lentísimo. - Olvidar que `e` debe ser **coprimo con φ(n)**: si no lo es, **no existe** `d`. - **Ignorar el *padding*.** Con `c = m^e` pelado, RSA es **determinista** y vulnerable: si cifrás el mismo mensaje dos veces, sale el mismo `c`; y con padding aleatorio (PKCS#1 v1.5 u OAEP) se evita.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2010-02, Pregunta 4 (8 puntos) — firma digital

> **Fuente:** Examen `sirc1002`, 16 de diciembre de 2010. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1002.pdf **Texto del examen: a)** Explique el concepto de Firma Digital y su uso en el caso de mensajes de correo electrónico. **b)** ¿MAC (Message Authentication Code) puede ser utilizado como técnica para firmar digitalmente? Justifique su respuesta.

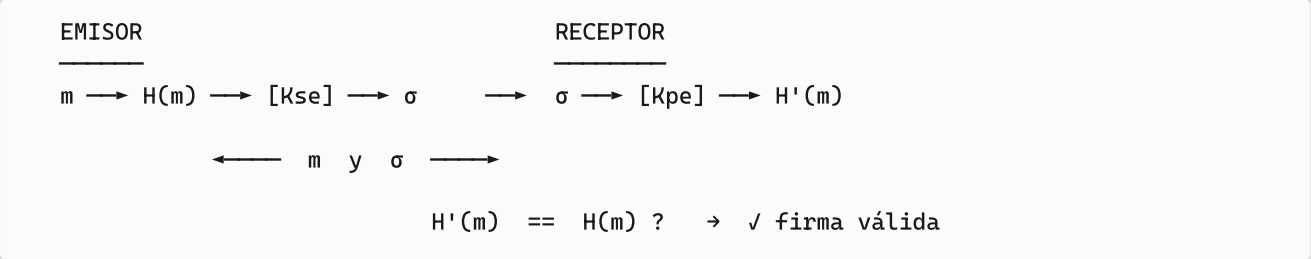
Resolución — parte a) (5 puntos)

La definición (1 punto). Una **técnica criptográfica** que busca lograr, en el mundo digital, objetivos **similares** a los de las firmas manuscritas tradicionales. Los cuatro: **verificable**, **no falsificable**, **no repudiable**, e **integridad del mensaje**.

El mecanismo, paso a paso (3 puntos) — esto es lo que hay que escribir:

La firma digital **combina criptografía asimétrica con funciones de hash**:

1. Del mensaje a firmar `m` se obtiene un hash: `H(m)`.
2. Ese hash **se cifra con la clave privada del emisor**: `σ = Kse(H(m))`.
3. Se envía **el mensaje `m` junto con la firma `σ`**. Y acá está la observación clave que la solución oficial underline y que hay que repetir: **no se movementa servicio de confidencialidad** — el mensaje va en claro.
4. El receptor calcula `H(m)` por su cuenta, descifra la firma con la clave **pública** del emisor, y **compara**.



Para qué sirve en el correo (1 punto). En un correo hay **cuatro** participantes que generan problemas de confianza distintos:

```
ANA ──escribe──►  [MDA de ana]  ──SMTP──►  [MTA del servidor de Beto]
                                                    |
                                          Beto podría leer y modificar
                                          el mensaje en el salto.
                                          PÉRDIDA DE CONFIDENCIALIDAD.

Y ADEMÁS:
ANA dice "yo escribí esto"          → AUTENTICIDAD (firma)
El mensaje llegó entero              → INTEGRIDAD
ANA no puede decir después           → NO REPUDIO
  "yo no lo escribí"
```

Y la respuesta clave, que es la que da el punto: *la firma digital no cifra el correo: el mensaje viaja en claro. Lo que hace es probar quién lo escribió y que no fue alterado. Para confidencialidad hace falta cifrado (`PGP` o `S/MIME`), que es otra cosa.*

Resolución — parte b) (3 puntos)

La respuesta es NO, y la justificación tiene que dar los dos pasos del argumento:

1. **El MAC se basa en criptografía simétrica.** Para verificar, hace falta la misma clave con la que se generó. Si Beto tiene que poder verificar la firma de Ana, hay que darle **la clave de Ana**.
2. **Si todos tienen la clave, todos pueden "firmar".** En el momento en que le das la clave a Beto para que pueda verificar, **Beto puede usarla para firmar mensajes haciéndose pasar por Ana**. No hay forma de que la clave sirva para verificar *y no* para firmar.

Y la extensión que suma puntos: la firma digital necesita que el receptor pueda **verificar** (con `kpe`, pública) pero **solo el emisor pueda firmar** (con `kse`, privada). **Esa asimetría es imposible con claves simétricas.** Con simetría, todo es simétrico: quien verifica, puede firmar.

> **Frase de cierre para sumar puntos:** *"el MAC da integridad y autenticación, pero no no-repudio: con una clave compartida, el no-repudio es imposible, porque todos los que pueden verificar pueden también firmar."*

3.2 Examen 2009-12, Pregunta 4 (6 puntos) — funciones de hash

> **Texto del examen: a)** Defina funciones de hash. **b)** Enumere las propiedades básicas de seguridad en la red, y especifique cuál o cuáles aporta la utilización de estas funciones. **c)** Mencione un ejemplo de función de hash comúnmente utilizada para intercambio de mensajes.

Resolución

a) Definición (2 puntos). Una **función de hash** es una función matemática que **transforma un mensaje de cualquier longitud** en una cadena de **longitud fija**, llamada **hash** o **resumen** o **huella**, tal que:

- es **fácil de calcular**;

- es **prácticamente imposible** recuperar el mensaje original a partir del hash (**unidireccional**);
- es **prácticamente imposible** encontrar dos mensajes distintos con el mismo hash (**resistente a colisiones**);
- **cualquier modificación, por mínima que sea, del mensaje produce un hash completamente diferente** (**efecto avalancha**).

b) Propiedades de seguridad (3 puntos) — esta es la parte que más puntos da:

Las cuatro propiedades de seguridad (**confidencialidad**, **integridad**, **autenticidad**, **no repudio**) y el aporte del hash:

Propiedad	¿El hash la aporta?	Razón
Confidencialidad	**NO**	El hash **no oculta nada**: es público, unidireccional y sin clave. Se obtiene el hash **exactamente igual** que el mensaje: solo sealary.
Integridad	**SÍ**	Si el mensaje se altera, el hash cambia. Quien recibe puede **recalcular** el hash y **comparar**. Es el aporte central.
Autenticidad	**NO** (por sí solo)	**Cualquiera** puede calcular el hash de cualquier mensaje. No requiere clave ni identidad.
No repudio	**NO**	Misma razón: cualquiera puede "firmar" (= hashear) cualquier mensaje. No hay no-repudio.

> **La conclusión, que es lo que la cátedra quiere que entiendas:** *el hash, por sí solo, aporta únicamente integridad. No aporta confidencialidad porque no cifra; no aporta autenticidad ni no repudio porque no tiene clave: cualquiera puede calcularlo. Para conseguir los otros tres hay que combinarlo con cifrado simétrico (autenticidad) o con cifrado asimétrico (autenticidad y no repudio).*

c) Ejemplo (1 punto). SHA-256, actualmente el estándar de facto (RFC 6234), y **SHA-3** (Keccak, FIPS 202). Anteriormente se usaban **MD5** y **SHA-1**, **ambos rotos**: MD5 por colisiones desde 2004 (y el *rogue CA* en 2008), SHA-1 por **SHAttered** en 2017. **Nunca usarlos.**

Lo que la solución oficial no dice y suma

El detalle del *fingerprint* en la práctica. Cuando hacés `sha256sum archivo`, lo que obtenés **no es** el hash del archivo entero: es el hash del **hash** (concatenado) de los archivos binarios que lo componen, más un sufijo con el **nombre y la longitud**. En Wireshark o en un certificado, el **fingerprint** del certificado es justamente `SHA-1` o `SHA-256` del DER codificado. Es un uso directo de la propiedad de *integridad*: si el fingerprint que vos anotaste a mano coincide con el que muestra el sistema, **nadie te cambió el certificado en tránsito**.

3.3 Examen 2009-02, Pregunta 1 (9 puntos) — TLS

> **Texto del examen: a)** Mencione el protocolo descrito durante el curso que puede cumplir con dicho objetivo [proteger a nivel de transporte una comunicación http, identificado como https]. **b)** Indique

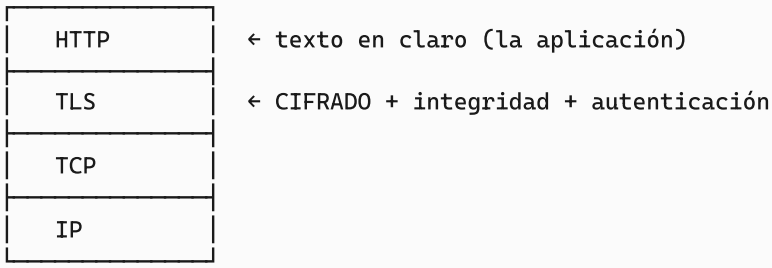
> **qué clave** contiene el certificado que el servidor le envía al cliente, y explique **para qué se utiliza** dicha clave. **c)** ¿**Qué clave/s** se utiliza/n para **proteger los datos** que viajan en el sentido **servidor → cliente**? **d)** Mencione otros componentes del certificado, y qué **elemento adicional** se necesita en el cliente para **asegurar la identidad del servidor**.

Resolución — la pregunta que hay que leer muy bien

> ⚠️ **Leelo dos veces antes de responder.** La (c) es la pregunta **tramposa** del examen: dice **"proteger los datos que viajan"**, no "proteger los datos". Y la diferencia entre esas dos frases es **toda la diferencia entre el examen de 2010 y el de 2026**.

a) El protocolo (1 punto)

TLS (*Transport Layer Security*), y su antecesor **SSL**. Se ubica **entre la capa de aplicación y la capa de transporte** (es decir, **capa 5-6** de TCP/IP): recibe los datos de HTTP, los cifra, y los pasa a TCP como si fueran datos normales.



b) Qué clave contiene el certificado (3 puntos)

Contiene la clave PÚBLICA del servidor, y sirve para que el cliente **pueda verificar que la clave privada que va a usar es realmente del servidor**.

Y para explicar "para qué se utiliza", hay que decir **las dos cosas**:

1. **Autenticar al servidor** ante el cliente: solo el servidor tiene la clave privada que corresponde a esa `kp`. Si el servidor puede firmar con la privada, es porque es el dueño de la pública del certificado. **Eso descarta el man-in-the-middle**, que es el propósito de TLS en el navegador.

> **El ataque que esto evita:** sin verificación de identidad, un atacante en el medio (café, wifi del aeropuerto, ISP) puede responder como si fuera el banco, y vos le mandás tu contraseña **a él**. TLS lo que hace es **obligar al atacante a tener la clave privada**, que no tiene.

2. **En las versiones antiguas, cifrar el *pre-master secret*.** En TLS 1.2 y anteriores, el cliente generaba el *pre-master secret* y lo cifraba con la `kp` del servidor: así **solo el servidor** podía descifrarlo. **Eso es lo que ya no se hace en TLS 1.3** (ver §Actualización).

c) Qué clave protege los DATOS (2 puntos)

Acá está la clave del examen. La clave que **realmente cifra los datos de aplicación** (los paquetes HTTP) es una **clave de sesión simétrica**, y en TLS 1.3 es siempre **(EC)DH key agreement** — es decir, **una clave compartida negociada, nunca enviada**.

```
La cadena completa, en TLS 1.3:

1. El servidor manda su CERTIFICADO con kp_srv.
2. El servidor manda su clave pública de (EC)DHE (key share).
3. El cliente manda su propia clave pública de (EC)DHE.
4. Ambos calculan el SECRETO COMPARTIDO = f(kp_srv_priv, kp_cli_pub)
   = f(kp_cli_priv, kp_srv_pub)          ← es el MISMO, por matemática
5. DerivAN la clave de sesión con un KDF (HKDF).
6. La clave de sesión cifra TODOS los datos HTTP, en los DOS sentidos.

★ La clave del certificado NO cifra los datos. Solo autentica al servidor.
★ La clave de sesión NO se envía nunca. Se acuerda, no se transmite.
```

> **La respuesta completa para el (c):** *la clave que protege los datos es la clave de sesión simétrica (derivada del secreto compartido de (EC)DH). La clave pública del certificado no cifra los datos: se usa para autenticar al servidor (o, en TLS ≤1.2, para cifrar el* pre-master secret*). En ambos sentidos de la comunicación se usa la misma clave de sesión, pero en general claves distintas para cada dirección (derived keys del HKDF), de modo que el tráfico de ida no se puede "desenmascarar" inyectando tráfico de vuelta.*

d) Otros componentes del certificado (3 puntos)

Los componentes que hay que enumerar:

Componente	Qué es
Versión y **Número de serie**	único por autoridad
Algoritmo de firma	con qué se firmó (SHA-256 con RSA, p. ej.)
Emisor (*issuer*)	quién firmó
Período de validez	*not before / not after*
Sujeto (*subject*)	CN, O, OU, C, L…
Clave pública del sujeto	← la del punto (b)
Huella digital (*fingerprint*)	hash del certificado
Extensiones	**SAN**, key usage, basic constraints, CRL distribution points

El elemento adicional que se necesita en el cliente (esto es lo que el examen pide y es la mitad del punto):

> **El almacén de certificados de confianza (el *trust store*), que contiene las claves públicas de las**

> **autoridades certificadoras raíz en las que el cliente confía.**

Sin ese almacén, el cliente **no tiene contra quién comparar** la firma del certificado: no puede verificar que quien firmó es una CA real. **Todo el modelo descansa en ese supuesto.**

La cadena completa, entonces: el certificado dice "*esta clave pública es de* `www.miuy.edu.uy`". El trust store dice "*esta CA es real*". Y la cadena de certificados **intermedios** une los dos extremos. Si falta cualquiera de los dos —o si no se valida el **SAN**— la conexión falla.

> ⚠️ **Nota importante sobre el (c) para 2026.** Si te examined en 2011, la respuesta esperada era "**RSA**" en el caso de TLS 1.2, porque la clave pública del certificado **cifraba el pre-master secret**. En TLS 1.3 eso **ya no existe**: **el transporte de clave con RSA fue eliminado** y todo pasa por (EC)DHE. Lo que **no cambia** es la idea: **la clave del certificado autentica, la clave de sesión cifra**. Si el examen te pide el nombre del algoritmo y estás en 2026, contestá **"(EC)DH (Diffie-Hellman) con X25519"** y aclará **"en TLS 1.2 era RSA"**.

3.4 Examen 2008-12, Pregunta 2 (6 puntos) — hub vs. switch y seguridad

Resolución. El enunciado compara **hub** y **switch** en cuanto a rendimiento y **seguridad**. La parte de seguridad es la que se pregunta y tiene tres respuestas concretas:

1. **El switch es un dispositivo de filtrado.** Recibe la trama completa en un puerto, mira la **MAC de destino**, y la manda **solo** por el puerto correspondiente. El hub, en cambio, la reenvía a **todos los puertos**.
2. **Aislamiento de dominios de colisión y de tramas.** Cada puerto del switch es un **dominio de colisión separado**, y solo el dominio del destino recibe la trama. El hub es **un solo** dominio de colisión.
3. **Sniffing.** Con el hub, cualquiera que enchufe un **sniffer** en cualquier puerto ve **todo** el tráfico. Con el switch, el sniffer solo ve el tráfico dirigido a su MAC (salvo **ataques de inundación de MAC**, que llenan la tabla de ARP/CAM y fuerzan al switch a retransmitir como un hub — que es precisamente por lo que existe **port security**).

Y el remate que hay que decir porque es lo que se pregunta en la cátedra: *el problema real no es que el hub no filtre: es que un switch no es una frontera de seguridad. Los tres modos de un switch —store-and-forward, cut-through, fragment-free— son decisiones de performance, no de seguridad. La "seguridad" de un switch es cero; todo lo que hace es optimizar el envío de tramas. Quien decide seguridad es el firewall, o el VLAN + ACL, no el switch.*

PARTE 4 — CIERRE

📌 Resumen en 10 líneas

1. **Confidencialidad** = ¿puede leerlo? · **Integridad** = ¿puede cambiarlo? · **Autenticidad** = ¿es de verdad? · **No repudio** = ¿podés probarlo?
2. Son **servicios independientes**: confidencialidad **no** implica integridad.
3. **Hash** (SHA-256): determinista, unidireccional, sin colisiones, avalancha. **Aporta integridad y nada más** — ni confidencialidad, ni autenticidad, ni no repudio, porque **no lleva clave**.
4. **MAC** (HMAC): clave simétrica. Aporta integridad + autenticación, pero **no confidencialidad** (no cifra) **ni no repudio** (todos pueden firmar).
5. **Firma digital**: hash + clave privada. Aporta integridad + autenticidad + **no repudio**, y **no aporta confidencialidad** (no cifra el mensaje).
6. **RSA**: `n = p·q`, `ϕ(n) = (p−1)(q−1)`, `e·d ≡ 1 mod ϕ(n)`. Lento, bloques de 245 bytes con clave de 2048 bits, y `PKCS#1 v1.5` **frágil**: hoy solo **RSA-PSS**.
7. **ECC**: misma seguridad con claves 12× más cortas. **ECDSA** firma, **(EC)DH** acuerda, **Ed25519** firma y es el default de SSH.
8. **Esquema híbrido**: clave de sesión aleatoria + AES-GCM para los datos, KEM para la clave. Por eso todo protocolo moderno cifra datos con simétrico.
9. **PKI**: la clave del certificado **autentica**, la clave de sesión **cifra**, y el **SAN** (no el CN) es lo que valida la identidad. Todo descansa en el **trust store**.
10. **2026**: RSA key transport eliminado en TLS 1.3; ML-KEM híbrido post-cuántico; SHA-1 **formalmente retracted**; y lo más importante: **cada ámbito tiene su ancla de confianza** (PKI, DNSSEC, RPKI, 802.1X, PGP).

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. ¿Por qué no se puede usar un hash como firma digital? Y ¿por qué no se puede usar un MAC?

<details>
<summary>Respuesta</summary>
</details>

2. ¿Por qué se firma el hash y no el mensaje?

<details>
<summary>Respuesta</summary>
</details>

3. En TLS 1.3, ¿qué clave protege los datos de aplicación? ¿Y para qué sirve la clave del certificado?

<details>
<summary>Respuesta</summary>
</details>

4. Calculá `d` para `p = 61`, `q = 53`, `e = 17`, y cifrá `m = 65`.

▶ Respuesta

5. ¿Por qué el SAN reemplazó al CN en los certificados?

▶ Respuesta

6. ¿Qué pasa si el *trust store* de tu máquina tiene una raíz falsa?

▶ Respuesta

🔗 Referencias

- **Examen** `sirc1002` — Pregunta 4 completa (8 pts: firma digital y por qué MAC no firma)
- **Examen** `sirc0912` — Pregunta 4 (6 pts: funciones de hash y propiedades de seguridad)
- **Examen** `sirc0902` — Pregunta 1 (9 pts: TLS, la clave del certificado y la clave de sesión)
- **Examen** `sirc0812` — Pregunta 2 (hub vs. switch: rendimiento y seguridad)
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 8 (seguridad)
- `RFC 2104` — HMAC · `RFC 8017` — PKCS #1 v2.2 (RSA) · `RFC 8017` **§8** — RSA-PSS · `RFC 5756` — política de validación X.509 · `RFC 5280` — PKI X.509 · `RFC 6962` — Certificate Transparency · `RFC 8446` — TLS 1.3 · `RFC 9155` — deprecación de MD5 y SHA-1 · `RFC 8709` — Ed25519 en SSH · `RFC 9421` — HTTP Message Signatures · `RFC 9580` — OpenPGP · `RFC 10024` — ML-KEM híbrido
- `ACTUALIZACIONES.md` **§13** — el cuadro de anclas de confianza por ámbito
- **Clase siguiente:** 48 — Seguridad aplicada (TLS, PGP, VPN, IDS/IPS, DPI) + IP Móvil + Simulacro final